

TÀI LIỆU HƯỚNG DẪN THỰC HÀNH

JAVA I/O STREAMS

Đọc - ghi dữ liệu dạng byte và ký tự

Thành phần	Nội dung
Đối tượng	Sinh viên đã học Java cơ bản và lập trình hướng đối tượng
Thời lượng đề xuất	3 tiết thực hành (150 phút)
Nền tảng	JDK 17 trở lên
Trọng tâm	java.io, java.nio.file, UTF-8, try-with-resources

1. Mục tiêu học tập

Sau khi hoàn thành bài thực hành, sinh viên có thể:

1. Giải thích được khái niệm luồng nhập, luồng xuất và điểm kết thúc luồng (EOF).
2. Phân biệt byte stream với character stream và chọn đúng loại luồng theo dữ liệu.
3. Đọc dữ liệu từ bàn phím bằng `BufferedReader`.
4. Đọc, ghi tệp văn bản UTF-8 và sao chép tệp nhị phân.
5. Sử dụng buffer, try-with-resources và xử lý `IOException` đúng cách.
6. Vận dụng I/O vào một chương trình OOP quản lý danh sách sản phẩm.

2. Yêu cầu chuẩn bị

Thành phần	Yêu cầu
Phần mềm	JDK 17+; IntelliJ IDEA, Eclipse hoặc VS Code có Java Extension Pack.
Kiến thức	Biến, vòng lặp, mảng, lớp/đối tượng, ngoại lệ và List.
Thư mục	Tạo project <code>JavaIOStream</code> với thư mục <code>src</code> và <code>data</code> .
Tệp thử nghiệm	Một tệp văn bản UTF-8 và một ảnh bất kỳ để kiểm tra sao chép nhị phân.

3. Kiến thức nền cần nhớ

Lưu ý thuật ngữ: Bài này học luồng vào/ra trong `java.io`. Đây không phải Stream API trong `java.util.stream` dùng để xử lý tập dữ liệu theo kiểu map, filter và reduce.

Loại dữ liệu	Lớp nền tảng	Lớp thường dùng	Ví dụ
Byte	<code>InputStream</code> / <code>OutputStream</code>	<code>BufferedInputStream</code> / <code>BufferedOutputStream</code>	Ảnh, PDF, ZIP, âm thanh
Ký tự	<code>Reader</code> / <code>Writer</code>	<code>BufferedReader</code> / <code>BufferedWriter</code> / <code>PrintWriter</code>	TXT, CSV, JSON, văn bản

3.1. Chuỗi chuyển đổi khi đọc văn bản

```
System.in (byte)
-> InputStreamReader (giải mã UTF-8)
-> BufferedReader (đọc hiệu quả, hỗ trợ readLine())
```

3.2. Các nguyên tắc quan trọng

1. `read()` hoặc `read(buffer)` trả về -1 khi đã đến cuối luồng.
2. Không dùng `available()` để xác định toàn bộ dữ liệu còn lại hoặc điểm kết thúc luồng.
3. Luôn chỉ rõ charset khi chuyển đổi giữa byte và ký tự; ưu tiên `StandardCharsets.UTF_8`.

4. Dùng try-with-resources cho file stream để tự động đóng tài nguyên ngay cả khi có lỗi.
5. Không đóng System.in hoặc System.out nếu chương trình còn cần sử dụng các luồng hệ thống này.

4. Cấu trúc project đề xuất

```

JavaIOStream/
├── src/
│   ├── ConsoleReaderDemo.java
│   ├── TextFileDemo.java
│   ├── BinaryFileCopy.java
│   ├── Product.java
│   └── ProductCsvApp.java
└── data/
    ├── ghi_chu.txt
    ├── products.csv
    └── source.jpg
  
```

Quy ước: Nếu chạy từ IDE, thư mục làm việc thường là thư mục gốc của project. Hãy kiểm tra working directory khi chương trình báo không tìm thấy tệp.

5. Thực hành 1 - Đọc văn bản từ bàn phím

5.1. Yêu cầu

Viết chương trình đọc từng dòng từ bàn phím. Chương trình dừng khi người dùng nhập q hoặc khi gặp EOF, đồng thời đếm số dòng hợp lệ đã nhập.

5.2. Mã nguồn tham khảo

```

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.nio.charset.StandardCharsets;

public class ConsoleReaderDemo {
    public static void main(String[] args) {
        BufferedReader reader = new BufferedReader(
            new InputStreamReader(System.in, StandardCharsets.UTF_8));

        int count = 0;
        System.out.println("Nhập văn bản; nhập q để kết thúc:");

        try {
            while (true) {
                String line = reader.readLine();

                if (line == null || line.equalsIgnoreCase("q")) {
                    break;
                }

                count++;
                System.out.printf("Dòng %d: %s\n", count, line);
            }
        } catch (IOException e) {
            System.err.println("Không thể đọc dữ liệu: " + e.getMessage());
        }

        System.out.println("Tổng số dòng đã nhập: " + count);
    }
}
  
```

5.3. Kiểm thử

Dữ liệu nhập	Kết quả cần quan sát
Xin chào Java	Hiển thị “Dòng 1: Xin chào Java”.
Dữ liệu tiếng Việt	Hiển thị đúng dấu tiếng Việt.
q	Thoát vòng lặp và in tổng số dòng.

5.4. Câu hỏi suy luận

1. Vì sao cần InputStreamReader giữa System.in và BufferedReader?
2. readLine() trả về giá trị nào khi gặp EOF?
3. Vì sao ví dụ không đóng reader gắn với System.in?

6. Thực hành 2 - Ghi và đọc tệp văn bản UTF-8

6.1. Yêu cầu

Tạo thư mục data, ghi ba dòng tiếng Việt vào data/ghi_chu.txt, sau đó đọc lại từng dòng và đánh số thứ tự.

6.2. Mã nguồn tham khảo

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

public class TextFileDemo {
    public static void main(String[] args) {
        Path file = Path.of("data", "ghi_chu.txt");

        try {
            Files.createDirectories(file.getParent());

            try (BufferedWriter writer = Files.newBufferedWriter(
                file, StandardCharsets.UTF_8)) {
                writer.write("Java I/O làm việc với các luồng dữ liệu.");
                writer.newLine();
                writer.write("BufferedWriter giúp ghi văn bản hiệu quả.");
                writer.newLine();
                writer.write("UTF-8 hỗ trợ tiếng Việt ổn định.");
            }

            try (BufferedReader reader = Files.newBufferedReader(
                file, StandardCharsets.UTF_8)) {
                String line;
                int number = 1;
                while ((line = reader.readLine()) != null) {
                    System.out.printf("%d. %s\n", number++, line);
                }
            }
        } catch (IOException e) {
            System.err.println("Lỗi xử lý tệp " + file + ": "
                + e.getMessage());
        }
    }
}
```

6.3. Kết quả mong đợi

1. Java I/O làm việc với các luồng dữ liệu.
2. BufferedWriter giúp ghi văn bản hiệu quả.
3. UTF-8 hỗ trợ tiếng Việt ổn định.

6.4. Nhiệm vụ mở rộng

1. Thay chế độ ghi mới bằng StandardOpenOption.CREATE và StandardOpenOption.APPEND để nối thêm nội dung.
2. In đường dẫn tuyệt đối của tệp bằng file.toAbsolutePath().
3. Thử đọc cùng tệp bằng charset khác và nhận xét kết quả.

7. Thực hành 3 - Sao chép tệp nhị phân

7.1. Yêu cầu

Sử dụng byte stream có bộ đệm để sao chép một ảnh. Tên tệp nguồn và tệp đích được truyền qua tham số dòng lệnh.

7.2. Mã nguồn tham khảo

```
import java.io.BufferedInputStream;
import java.io.BufferedOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.nio.file.Files;
import java.nio.file.Path;

public class BinaryFileCopy {
    private static final int BUFFER_SIZE = 8192;

    public static void main(String[] args) {
        if (args.length != 2) {
            System.out.println("Cách dùng: java BinaryFileCopy <nguồn> <đích>");
            return;
        }

        Path source = Path.of(args[0]);
        Path target = Path.of(args[1]);
        long totalBytes = 0;

        try (InputStream input = new BufferedInputStream(
            Files.newInputStream(source));
            OutputStream output = new BufferedOutputStream(
                Files.newOutputStream(target))) {

            byte[] buffer = new byte[BUFFER_SIZE];
            int bytesRead;

            while ((bytesRead = input.read(buffer)) != -1) {
                output.write(buffer, 0, bytesRead);
                totalBytes += bytesRead;
            }

            System.out.println("Đã sao chép " + totalBytes + " byte.");
        } catch (IOException e) {
            System.err.println("Sao chép thất bại: " + e.getMessage());
        }
    }
}
```

7.3. Biên dịch và chạy

```
javac -d out src/BinaryFileCopy.java
java -cp out BinaryFileCopy data/source.jpg data/copy.jpg
```

7.4. Tiêu chí kiểm thử

1. Tập copy.jpg được tạo và mở được bình thường.
2. Kích thước Files.size(source) và Files.size(target) bằng nhau.
3. Khi tệp nguồn không tồn tại, chương trình hiển thị lỗi và không bị dừng bất thường.

Điểm cần nhớ: write(buffer, 0, bytesRead) chỉ ghi đúng số byte vừa đọc. Không được luôn ghi toàn bộ buffer vì lần đọc cuối có thể chứa ít hơn 8192 byte.

8. Thực hành 4 - Quản lý sản phẩm bằng tệp CSV

8.1. Bài toán

Đọc danh sách sản phẩm từ data/products.csv, tạo các đối tượng Product, hiển thị danh sách, tính tổng giá trị tồn kho và ghi kết quả vào data/report.txt.

8.2. Dữ liệu mẫu

```
ma,ten,donGia,soLuong
SP01,Ban phim,350000,10
SP02,Chuot khong day,250000,5
SP03,Tai nghe,600000,3
```

8.3. Lớp Product

```
public class Product {
    private final String code;
    private final String name;
    private final double unitPrice;
    private final int quantity;

    public Product(String code, String name,
                   double unitPrice, int quantity) {
        if (code == null || code.isBlank()) {
            throw new IllegalArgumentException("Mã không được rỗng");
        }
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Tên không được rỗng");
        }
        if (unitPrice <= 0 || quantity < 0) {
            throw new IllegalArgumentException("Giá hoặc số lượng không hợp lệ");
        }
        this.code = code;
        this.name = name;
        this.unitPrice = unitPrice;
        this.quantity = quantity;
    }

    public double inventoryValue() {
        return unitPrice * quantity;
    }

    @Override
    public String toString() {
        return "%s - %s: %,0f VND".formatted(
            code, name, inventoryValue());
    }
}
```

8.4. Lớp ProductCsvApp

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;

public class ProductCsvApp {
    public static void main(String[] args) {
        Path input = Path.of("data", "products.csv");
        Path report = Path.of("data", "report.txt");
```

```

List<Product> products = new ArrayList<>();

try (BufferedReader reader = Files.newBufferedReader(
    input, StandardCharsets.UTF_8)) {
    reader.readLine(); // bỏ qua dòng tiêu đề
    String line;
    int lineNumber = 1;

    while ((line = reader.readLine()) != null) {
        lineNumber++;
        if (line.isBlank()) continue;

        String[] parts = line.split(",", -1);
        if (parts.length != 4) {
            System.err.println("Bỏ qua dòng " + lineNumber);
            continue;
        }

        try {
            products.add(new Product(
                parts[0].trim(), parts[1].trim(),
                Double.parseDouble(parts[2].trim()),
                Integer.parseInt(parts[3].trim()));
        } catch (IllegalArgumentException e) {
            System.err.println("Dòng " + lineNumber
                + " không hợp lệ: " + e.getMessage());
        }
    }
} catch (IOException e) {
    System.err.println("Không đọc được CSV: " + e.getMessage());
    return;
}

double total = 0;
for (Product product : products) {
    System.out.println(product);
    total += product.inventoryValue();
}

try (BufferedWriter writer = Files.newBufferedWriter(
    report, StandardCharsets.UTF_8)) {
    writer.write("Số sản phẩm: " + products.size());
    writer.newLine();
    writer.write("Tổng giá trị tồn kho: %,0f VND".formatted(total));
    writer.newLine();
} catch (IOException e) {
    System.err.println("Không ghi được báo cáo: " + e.getMessage());
}
}

```

Giới hạn CSV: Cách split bằng dấu phẩy chỉ phù hợp dữ liệu đơn giản, không xử lý tên chứa dấu phẩy trong dấu ngoặc kép. Với hệ thống thực tế nên dùng thư viện CSV chuyên dụng.

8.5. Kết quả mong đợi

```

SP01 - Bàn phím: 3,500,000 VND
SP02 - Chuột không dây: 1,250,000 VND
SP03 - Tai nghe: 1,800,000 VND

```

```

Nội dung report.txt:
Số sản phẩm: 3
Tổng giá trị tồn kho: 6,550,000 VND

```

9. Bài tập tổng hợp bắt buộc

Phát triển chương trình InventoryManager quản lý tồn kho bằng OOP và I/O streams.

9.1. Yêu cầu chức năng

4. Nhập danh sách sản phẩm từ bàn phím; mã và tên không rỗng, đơn giá lớn hơn 0, số lượng không âm.
5. Lưu danh sách vào data/inventory.csv bằng UTF-8.
6. Đọc lại tệp CSV và tái tạo danh sách đối tượng Product.
7. Hiển thị toàn bộ sản phẩm và tổng giá trị tồn kho.
8. Tìm sản phẩm có giá trị tồn kho cao nhất.
9. Ghi báo cáo tổng hợp vào data/inventory-report.txt.
10. Xử lý được tệp không tồn tại, dòng CSV thiếu cột và dữ liệu số không hợp lệ.

9.2. Quy định kỹ thuật

11. Thuộc tính lớp Product phải được đóng gói.
12. Dùng BufferedReader/BufferedWriter hoặc Files.newBufferedReader/newBufferedWriter.
13. Chỉ rõ StandardCharsets.UTF_8.
14. Dùng try-with-resources cho mọi tệp được mở.
15. Không dùng InputStream.available() để điều khiển vòng lặp đọc.
16. Thông báo lỗi phải chỉ rõ tệp hoặc dòng dữ liệu liên quan.

9.3. Bộ dữ liệu kiểm thử tối thiểu

Trường hợp	Dữ liệu	Kết quả mong đợi
Hợp lệ	3 sản phẩm đúng định dạng	Lưu, đọc lại và tính tổng chính xác.
Mã rỗng	Mã = chuỗi rỗng	Từ chối dữ liệu và báo nguyên nhân.
Giá âm	Đơn giá = -1000	Từ chối dữ liệu; chương trình tiếp tục hoạt động.
CSV lỗi	Một dòng chỉ có 3 cột	Bỏ qua dòng, báo số dòng lỗi.
Tệp thiếu	Đổi tên inventory.csv	Thông báo không tìm thấy tệp.
Tiếng Việt	Tên = Bàn phím cơ	Ghi và đọc lại đúng dấu.

9.4. Sản phẩm cần nộp

1. Các file .java và hai file dữ liệu đầu ra.
2. Ảnh chụp kết quả chạy với tối thiểu ba trường hợp kiểm thử.
3. Một đoạn nhận xét ngắn về byte stream, character stream, charset và try-with-resources.

10. Câu hỏi báo cáo

1. InputStream và Reader khác nhau như thế nào? Khi nào dùng mỗi loại?
2. Giá trị -1 của read() hoặc read(buffer) có ý nghĩa gì?
3. Vì sao available() không nên dùng để xác định đã đọc hết luồng?
4. InputStreamReader có vai trò gì trong chuỗi xử lý văn bản?
5. Buffer cải thiện hiệu năng đọc/ghi như thế nào?
6. Vì sao phải chỉ rõ UTF-8 khi xử lý dữ liệu tiếng Việt?
7. try-with-resources giải quyết vấn đề gì?
8. flush() khác close() như thế nào?
9. Vì sao tệp ảnh phải được xử lý bằng byte stream thay vì Reader/Writer?
10. Nếu read(buffer) trả về 120, vì sao chỉ được ghi 120 byte từ buffer?